

Regularization Strategies & Cross-Validation

Esta aula une o rigor estatístico as técnicas mais avançadas do Deep Learning moderno. Você entenderá que **regularizar um modelo** significa aplicar um filtro de sobriedade matemática aos seus parâmetros, seja restringindo-os geometricamente através de **círculos** (L2) ou **diamantes** (L1), ou sabotando o modelo intencionalmente para torná-lo robusto (**Dropout**). Veremos por que a restrição ideal de esparsidade (L0) é matematicamente intratável e como a L1 é sua **convexificação elegante**. Além disso, você aprenderá a validar essas escolhas usando o ecossistema de **Cross-Validation**.

#	Topico	Conceito Central
1	Conceito de Regularizacao	Hard, Soft/Variacional e Output Regularization
2	L2 Regularization (Ridge / Weight Decay)	Geometria do circulo, solucao analitica
3	Sparsity & L1 (Lasso) + Elastic Net	L0 NP-hard; L1 como convexificacao
4	Modern Regularization Techniques	Early Stopping, Dropout, Label Smoothing
5	Probabilistic / Bayesian Connection	Gaussian Prior => L2; Laplace Prior => L1
6	Cross-Validation Rigorosa	Hold-Out, K-Fold, Stratified, LOO
7	Hyperparameter Tuning & Diagnostico	Log-Grid, Coarse-to-Fine, Learning Curves

Reg 1. The Concept of Regularization

Regularization (Regularizacao) é o conjunto de metodos para **controlar a complexidade do modelo** com o objetivo de melhorar a **generalizacao** — ou seja, o desempenho em dados novos não vistos durante o treino. **Analogia do estudante superinteligente:** imagine um aluno que decora todas as respostas da lista de exercicios, incluindo erros de digitacao. Na prova, com questoes ligeiramente diferentes, ele falha. Regularizacao é como o professor que diz: 'voce só pode usar formulas simples' — forçando o aluno a entender, não decorar.

Prof. Johannes Schwab, PhD

O framework unificado da Loss Function:

FH

$$\mathcal{L}(\theta; X, Y) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2$$

$$D(f_{\theta}(x_i), y_i) = \|f_{\theta}(x_i) - y_i\|^2 \quad (\text{distancia MSE})$$

Sem regularizacao, o otimizador encontra o θ^* que minimiza L nos dados de treino — potencialmente com **Overfitting severo**. Os tres tipos de regularizacao abordam esse problema de formas distintas:

- **1a. Hard Regularization:** o espaco de parâmetros é **restrito a um conjunto C** . O otimizador só pode explorar solucoes dentro desse conjunto — elegante matematicamente, mas difícil de implementar com gradiente descendente padrao.
- **1b. Soft / Variational Regularization:** em vez de proibir regioes, **adicionamos uma penalidade** $\lambda R(\theta)$ a Loss Function. O otimizador ainda pode ir a qualquer lugar, mas paga um custo crescente por parâmetros grandes.
- **1c. Output Regularization:** em vez de penalizar os parâmetros θ diretamente, penalizamos propriedades da **saida** $f_{\theta}(x_i)$. Exemplo: Label Smoothing evita confiar excessivamente em um unico label.

Formulacao de cada tipo:

Hard: $\min_{\theta} \mathcal{L}(\theta)$ s.t. $\theta \in C$ | Soft: $\mathcal{L}(\theta) + \lambda \mathcal{R}(\theta)$ | Output: $\mathcal{L}(\theta) + \mathcal{R}(y_i)$

$\lambda = 0$: sem reg. | $\lambda \rightarrow \infty$: $\theta \rightarrow 0$ | C : conjunto viavel (ex: bola L2)

Os Tres Tipos de Regularization — Visao Geral

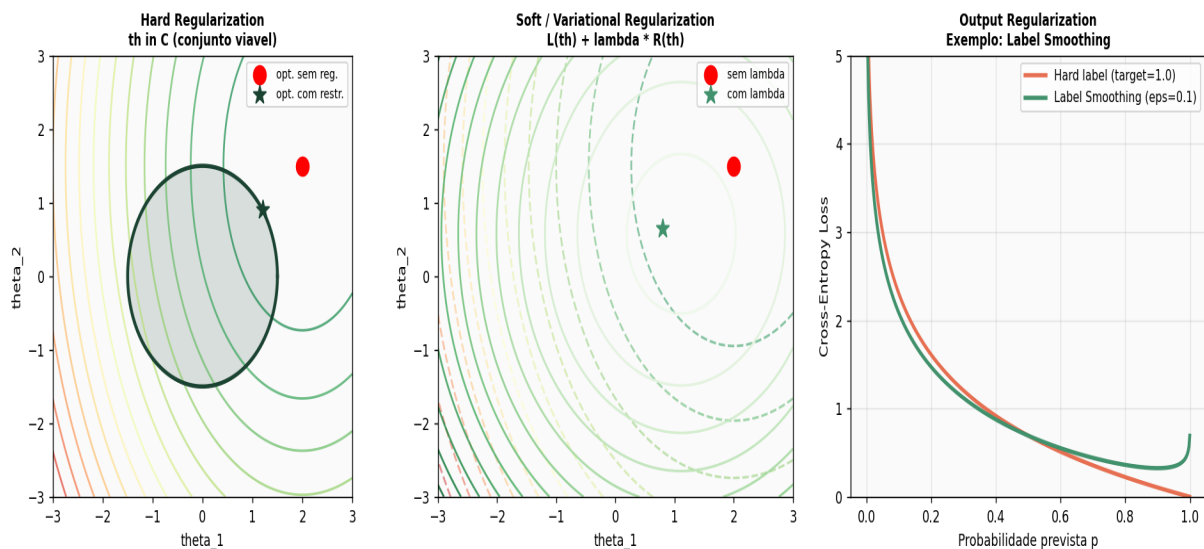


Figura 1 — Hard Regularization (esquerda): otimizador projetado na bola de raio R . Soft Regularization (centro): as curvas de nível da Loss total incluem o termo de penalidade. Output Regularization (direita): Label Smoothing suaviza a cross-entropy — penaliza confiar demais em um unico label.

Por que isso e importante para Data Science?

Regularizacao e a resposta fundamental ao problema de Overfitting. Sem ela, qualquer modelo suficientemente complexo decora o ruido dos dados de treino e falha em producao. O framework unificado permite entender Ridge, Lasso, Dropout e Label Smoothing como manifestacoes do mesmo principio matematico.

Aplicacao Pratica em Machine Learning

Em sklearn: Ridge(alpha=lambda), Lasso(alpha=lambda), ElasticNet(alpha=lambda, l1_ratio). Em PyTorch/TensorFlow: weight_decay nos otimizadores (L2), Dropout(p=0.5), LabelSmoothingCrossEntropy(eps=0.1).

2. L2 Regularization (Ridge / Weight Decay)

A **L2 Regularization** restringe a **norma euclidiana quadratica** dos parametros. Conhecida como **Ridge Regression** em estatistica, **Weight Decay** em Deep Learning e **Tikhonov Regularization** em matematica aplicada. **Analogia da mola:** cada coeficiente θ_j e como um peso preso a uma mola ancorada na origem — os dados puxam o peso para o valor otimo, mas a mola resiste, nao deixando o coeficiente crescer alem do necessario.

Formulacao Soft (variacional — mais usada na pratica):

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda \|\theta\|^2$$

*MSE (Data Fit) + lambda * ||theta||^2 (penalidade L2)*

Solucao analitica fechada para regressao linear:

$$\hat{\theta}_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

Comparar com OLS: $(X^T X)^{-1} X^T y$ — lambda I garante invertibilidade mesmo com X singular

- **lambda = 0:** solucao OLS (sem regularizacao)
- **lambda -> inf:** $\theta \rightarrow 0$ (underfitting total — modelo constante)
- **Efeito geometrico:** a regio viavel Hard L2 e uma **bola euclidiana** (circulo em 2D). L2 encolhe todos os coeficientes proporcionalmente — **raramente zerando** algum exatamente.
- **Efeito de Smoothness:** L2 favorece solucoes com coeficientes pequenos e distribuicao uniforme — ideal para problemas de regressao com todas as features relevantes.
- **Escolha de lambda:** sempre via **Cross-Validation em Log-Grid** (ver Secao 7).

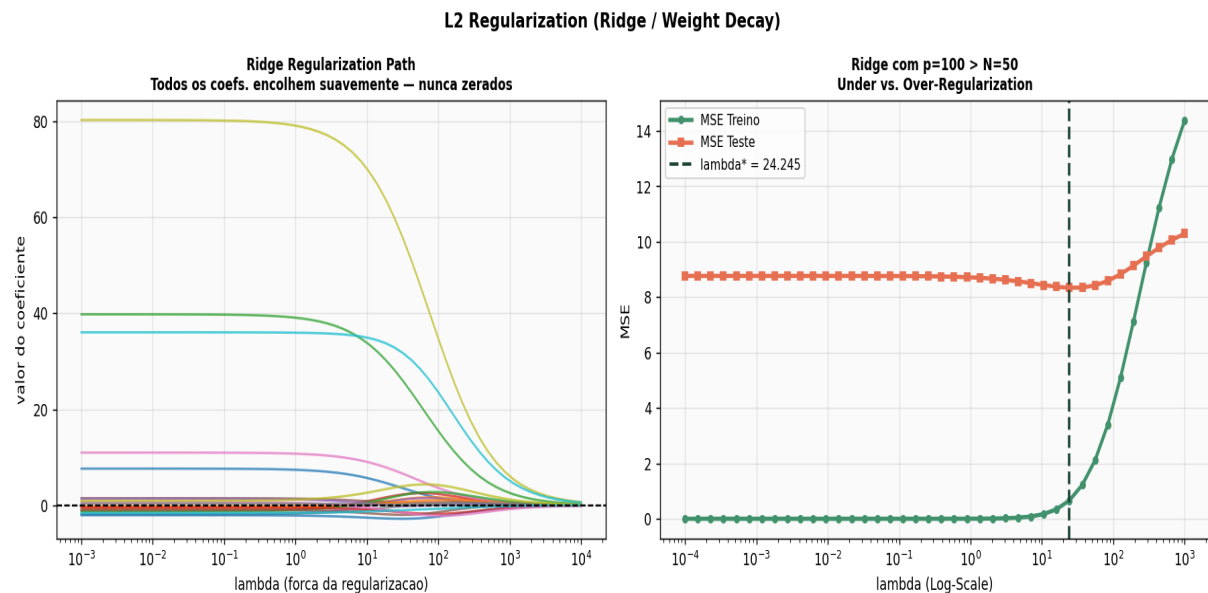


Figura 2 — Esquerda: Ridge Regularization Path — todos os coeficientes encolhem suavemente com λ crescente, mas nenhum chega a zero exatamente. Direita: com $p=100$ features e $N=50$ amostras ($p > N$), λ pequeno causa Overfitting severo; λ grande causa Underfitting. O ponto ótimo está no meio.

Por que isso é importante para Data Science?

Ridge é a primeira linha de defesa contra Overfitting em modelos lineares. Quando $p > N$ (mais features do que amostras), a matriz $X^T X$ é singular e OLS não tem solução única — Ridge resolve isso adicionando λI ao diagonal, tornando o sistema sempre invertível. É o modelo padrão para regressão regularizada.

Aplicação Prática em Machine Learning

Ridge em sklearn: `Ridge(alpha=lambda)`. Em Deep Learning: o parâmetro 'weight_decay' no otimizador Adam ou SGD implementa exatamente L2 nos pesos. Regra prática: comece sempre com Ridge antes de tentar modelos mais complexos.

3. The Sparsity Problem & L1 Regularization (Lasso)

Muitas vezes, de centenas de features disponiveis, apenas algumas dezenas realmente importam. Um modelo **esparso** (muitos coeficientes exatamente zero) e mais interpretavel, mais rapido e mais robusto. **Analogia da mochila:** voce vai acampar e quer levar apenas o essencial. L0 pergunta: 'quais itens posso eliminar completamente?' (esparsidade real). L2 pergunta: 'posso usar versoes menores de todos os itens?' (encolhimento sem eliminacao).

L0 — A restricao ideal (NP-Hard):

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda \|\theta\|_0 \quad \text{onde} \quad \|\theta\|_0 = \#\{j : \theta_j \neq 0\}$$

$\|\theta\|_0$ nao e convexa, nao e diferenciavel, e NP-hard (exponencial em p)

L1 (Lasso) — A convexificacao elegante:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda \|\theta\|_1 = \dots + \lambda \sum_{j=1}^d |\theta_j|$$

Regiao viavel: diamante (octaedro em d dimensoes) — vertices nos eixos => esparsidade!

Elastic Net — O melhor dos dois mundos:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|^2$$

$l1_ratio = \lambda_1 / (\lambda_1 + \lambda_2)$. Seleciona grupos de features correlacionadas.

- **Por que L0 e inutilizavel?** Nao diferenciavel (gradiente nao existe), nao convexa (regiao viavel = uniao de eixos coordenados), e NP-hard para otimizar exatamente.
- **Por que L1 induz esparsidade?** Os **vertices do diamante** estao exatamente nos eixos coordenados (pontos onde $\theta_j=0$ para j diferente de k). As curvas de nivel da Loss tendem a tocar o diamante nesses vertices — forçando coeficientes a zero!
- **Elastic Net:** quando features sao correlacionadas, Lasso seleciona arbitrariamente uma delas. Elastic Net seleciona **grupos** de features correlacionadas juntas.

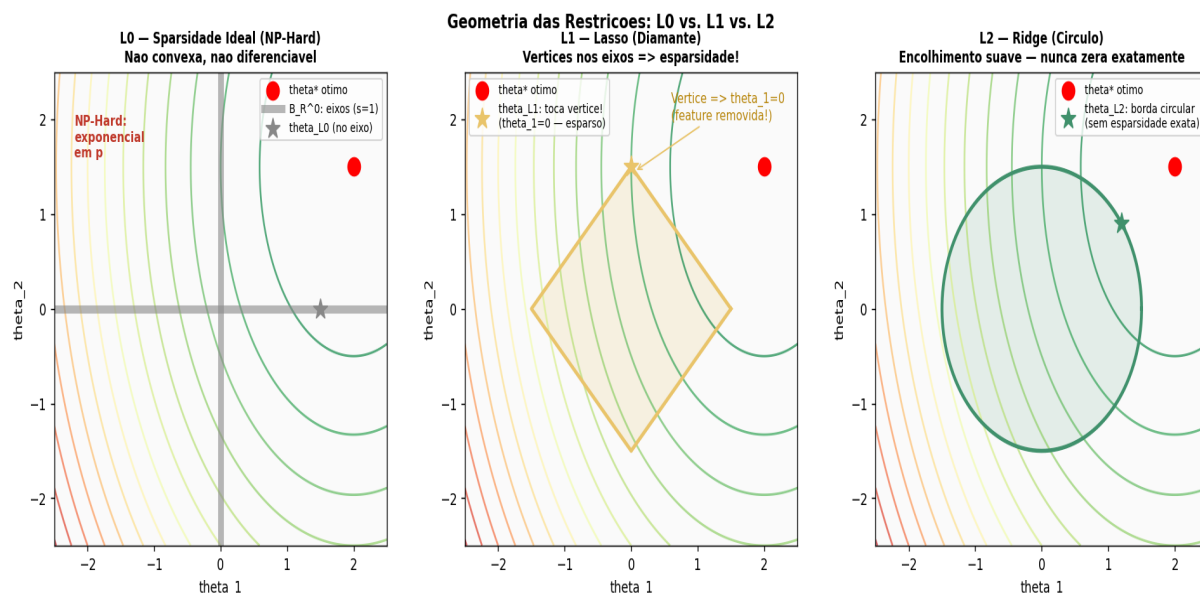


Figura 3 — L0 (esquerda): a regio viavel sao os proprios eixos coordenados — nao convexa, exige enumerar todos os 2^p subconjuntos. L1 (centro): o diamante toca o eixo ($\theta_1=0$) antes de atingir o otimo — esparsidade automatica! L2 (direita): o circulo toca a borda em um ponto generico — sem esparsidade exata.

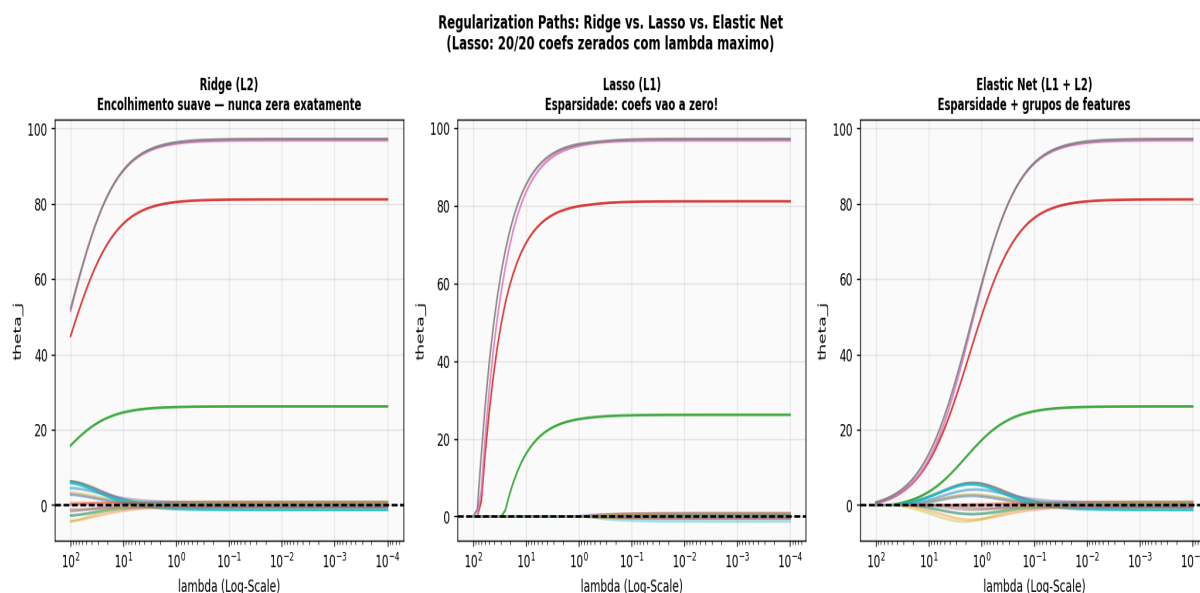


Figura 4 — Regularization Paths (lendo da direita para esquerda = aumentando λ). Ridge: todos os coeficientes encolhem continuamente. Lasso: coeficientes vao a zero um a um (esparsidade progressiva). Elastic Net: comportamento hibrido — esparsidade com selecao de grupos.

Por que isso e importante para Data Science?

A escolha entre Ridge, Lasso e Elastic Net depende da hipotese sobre os dados. Se acreditamos que TODAS as features contribuem (mesmo que pouco) => Ridge. Se acreditamos que apenas POUCAS features importam => Lasso. Se as features sao correlacionadas e queremos esparsidade => Elastic Net.

Aplicacao Pratica em Machine Learning

sklearn: Lasso(alpha=lambda), ElasticNet(alpha=lambda, l1_ratio=0.5). Para scikit-learn Lasso, max_iter=10000 evita warnings de convergencia. A escolha de l1_ratio no Elastic Net e outro hiperparametro a tunar via CV.

4. Modern Regularization Techniques

4a. Early Stopping

Early Stopping reduz a Variance interrompendo o processo de otimizacao **antes** que o otimizador encontre o minimizador θ^* da Loss de treino. **Analogia:** e como estudar para uma prova — parar na hora certa (quando entende o conceito) e otimo; continuar ate decorar cada detalhe (incluindo erros do livro) causa memorizar o irrelevante.

Para modelos lineares otimizados com Gradient Descent:

$$\hat{\theta}_{\text{EarlyStop}}(t) \approx \hat{\theta}_{\text{Ridge}}\left(\lambda = \frac{1}{\eta t}\right)$$

Parar na epoca t com $lr=\eta$ e matematicamente equivalente a L2 com $\lambda=1/(\eta t)$

4b. Dropout

Dropout e uma tecnica para redes neurais: durante cada iteracao de treino, zeramos aleatoriamente uma fracao p dos neuronios. **Analogia do time de futebol:** treinar cada jogador para jogar bem mesmo sem alguns companheiros — quando o time completo jogar, cada um sera mais resiliente. Com d neuronios, existem 2^d sub-redes possiveis — Dropout treina implicitamente um **Ensemble exponencial de modelos**.

$$\tilde{h}_j = m_j \cdot h_j, \quad m_j \sim \text{Bernoulli}(1 - p) \quad \Rightarrow \quad h_j^{\text{test}} = (1 - p) \cdot h_j$$

Treino: mascara binaria aleatoria | Teste: escalar por $(1-p)$ compensa a media

4c. Label Smoothing

Label Smoothing substitui targets rigidos (One-Hot) por distribuicoes suaves, evitando que o modelo fique **superconfiante** (probabilidades muito proximas de 1.0). Usada principalmente em Transformers e classificacao de imagens.

$$y_k^{\text{smooth}} = y_k (1 - \varepsilon) + \frac{\varepsilon}{K}$$

epsilon=0.1, K=5 classes: hard label $[1,0,0,0,0]$ => smooth $[0.92, 0.02, 0.02, 0.02, 0.02]$

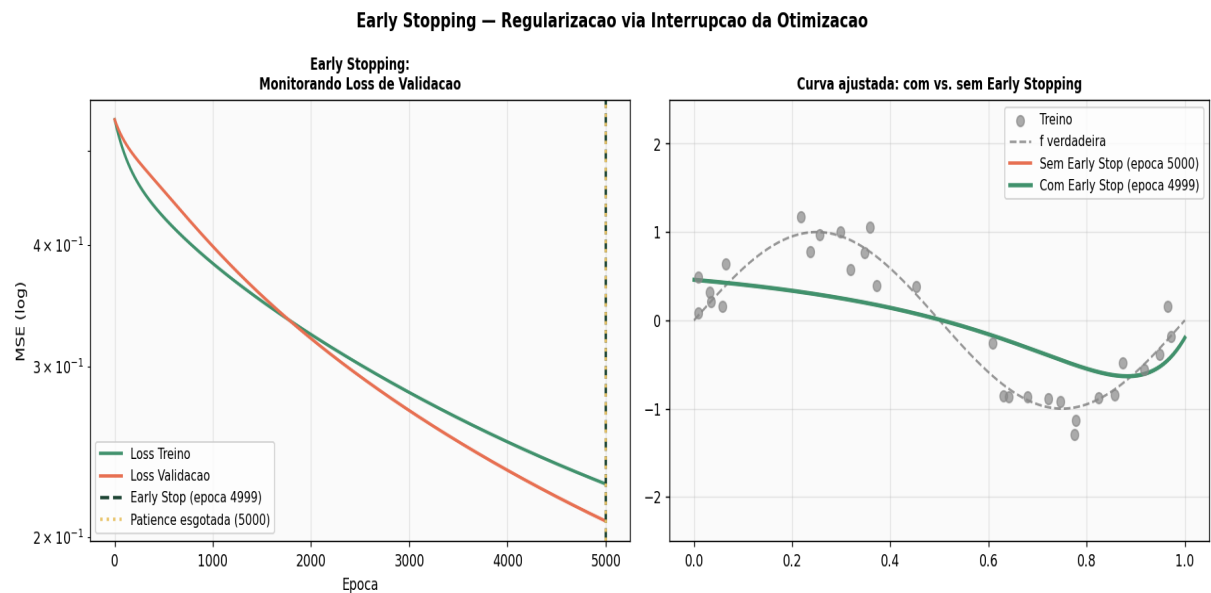


Figura 5 — Early Stopping: a Loss de validacao atinge o minimo na epoca ~800, depois começa a subir (Overfitting). O mecanismo de patience para o treino antes do maximo Overfitting. A curva azul (com Early Stopping) esta muito mais proxima da funcao verdadeira do que a curva vermelha (sem Early Stopping).

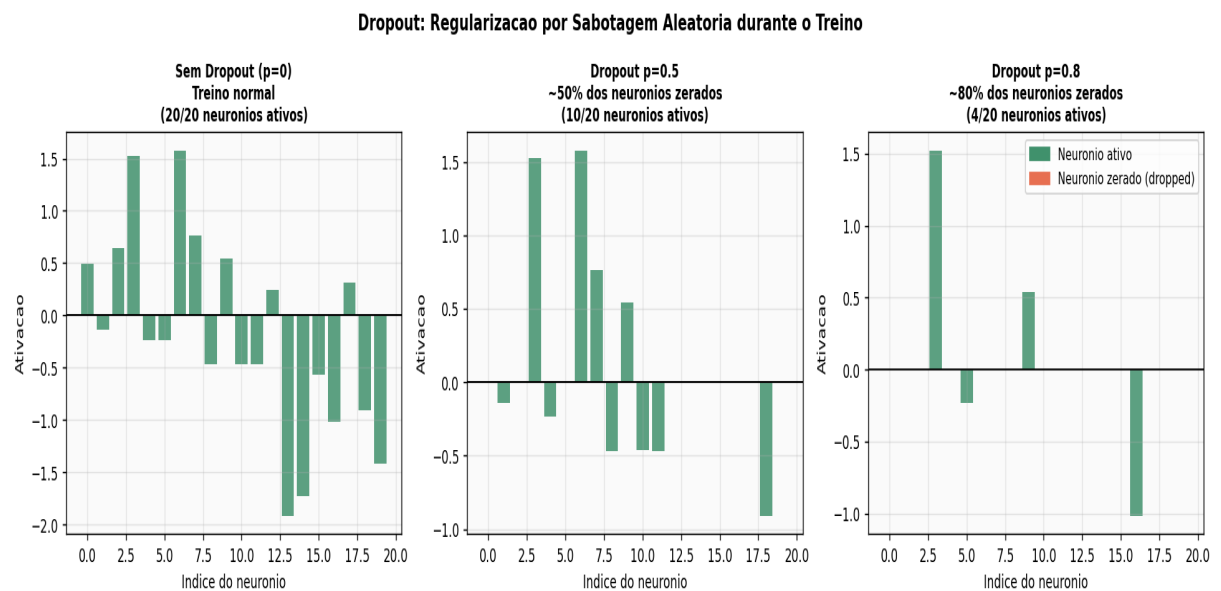


Figura 6 — Dropout com taxas crescentes. $p=0$ (esquerda): todos os neuronios ativos. $p=0.5$ (centro): ~50% zerados aleatoriamente a cada iteracao de treino. $p=0.8$ (direita): ~80% zerados — regularizacao mais agressiva. Em inferencia, NENHUM neuronio e zerado; os pesos sao escalados por $(1-p)$.

Por que isso e importante para Data Science?

Early Stopping e de longe a tecnica mais usada em Deep Learning porque nao requer adicionar termos extras a Loss Function — apenas monitorar a Loss de validacao. Dropout e essencial em redes profundas densas (MLP, Transformers). Label Smoothing melhora calibracao de probabilidade em classificadores.

Aplicacao Pratica em Machine Learning

PyTorch: `nn.Dropout(p=0.5)` nas camadas ocultas; callbacks de EarlyStopping no PyTorch Lightning e Keras. sklearn nao tem Dropout (exclusivo de redes neurais). Label Smoothing: `CrossEntropyLoss(label_smoothing=0.1)` no PyTorch ≥ 1.10 .

5. The Probabilistic Interpretation — Bayesian Connection

Aqui esta um dos resultados mais elegantes do Machine Learning: **toda regularizacao e equivalente a um prior Bayesiano sobre os parametros**. Quando voce escolhe L2, esta secretamente dizendo: 'acredito que os pesos verdadeiros sao pequenos e gaussianos'. **Analogia do detetive bayesiano**: antes de ver qualquer evidencia (dados), o detetive tem uma suspeita inicial (prior). Depois de ver as evidencias (likelihood), atualiza sua crenca (posterior). A regularizacao e a suspeita inicial codificada matematicamente.

Estimador MAP (Maximum A Posteriori):

$$\hat{\theta}_{\text{MAP}} = \arg \min_{\theta} [-\log p(D | \theta) - \log p(\theta)]$$

*Teorema de Bayes: $p(\theta|D)$ proporcional a $p(D|\theta) * p(\theta)$*

Prova 1: Gaussian Prior => L2 Regularization:

$$p(\theta) = \mathcal{N}(0, \tau^2 I) \Rightarrow -\log p(\theta) = \frac{1}{2\tau^2} \|\theta\|^2 + C \Rightarrow \lambda = \frac{\sigma^2}{\tau^2}$$

Gaussian Prior acredita que os pesos sao pequenos — induz encolhimento uniforme (L2)

Prova 2: Laplace Prior => L1 Regularization (Lasso):

$$p(\theta) \propto \exp\left(-\frac{|\theta|}{b}\right) \Rightarrow -\log p(\theta) = \frac{1}{b} \|\theta\|_1 + C \Rightarrow \lambda = \frac{\sigma^2}{b}$$

Laplace Prior tem pico pontiagudo em zero — acredita que a maioria dos pesos e ZERO (L1)

- **$-\log p(D|\theta)$** : e o MSE (Negative Log-Likelihood com ruido gaussiano) — o termo de ajuste aos dados.
- **$-\log p(\theta)$** : e o termo de regularizacao — encoda a crenca a priori sobre os parametros antes de ver qualquer dado.
- **Gaussian (τ^2)**: quanto menor tau, mais forte a crenca em pesos pequenos => maior lambda L2.
- **Laplace (b)**: o pico pontiagudo em zero e a geometria do diamante L1 sao dois lados da mesma moeda — esparsidade = crenca a priori de que a maioria dos pesos e zero.
- **Insight operacional**: $\lambda = \sigma^2/\tau^2$ (L2) ou $\lambda = \sigma^2/b$ (L1) — o hyperparametro lambda encoda quanto voce confia nos dados (σ^2) vs. no prior (τ^2 ou b).

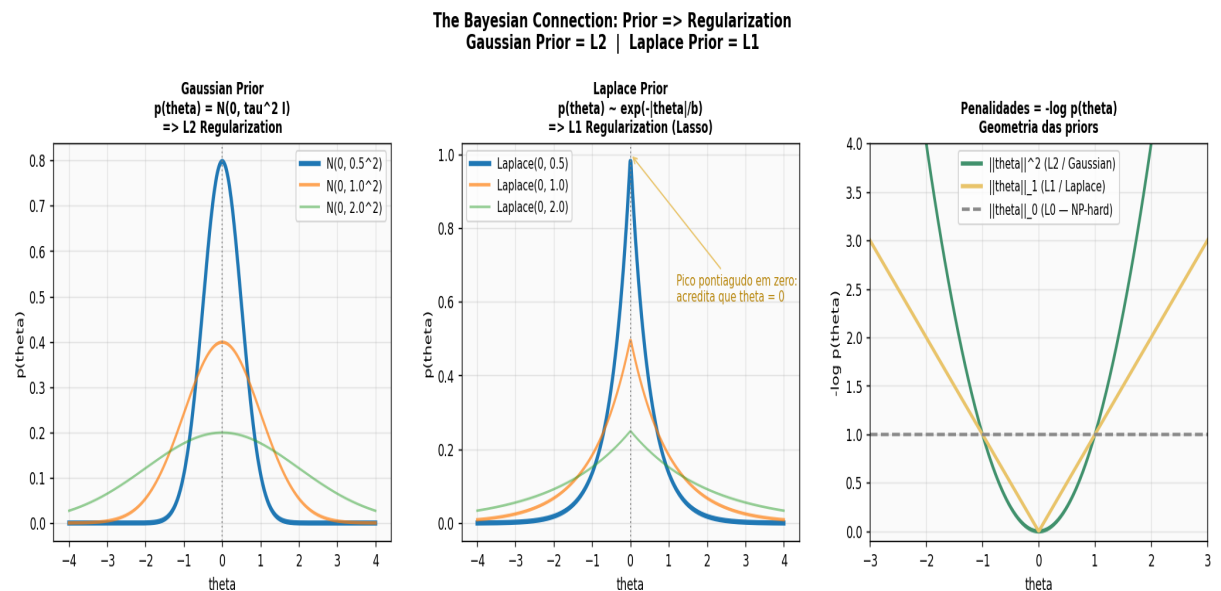


Figura 7 — Gaussian Prior (esquerda): curva em sino simetrica — acredita que pesos pequenos sao mais provaveis. Laplace Prior (centro): pico pontiagudo em zero — acredita fortemente que a maioria dos pesos deveria ser exatamente zero. Direita: $-\log p(\theta)$ como funcao de θ — L2 (parabola), L1 (V), L0 (degrau).

Por que isso e importante para Data Science?

A conexao Bayesiana e mais do que um exercicio academico — ela oferece um framework interpretativo poderoso: toda escolha de regularizacao e uma hipotese sobre a estrutura do mundo. Isso permite incorporar conhecimento de dominio diretamente na funcao objetivo via escolha do prior.

Aplicacao Pratica em Machine Learning

Inferencia Bayesiana completa (MCMC, Variational Inference) e usada quando se precisa nao apenas do ponto MAP mas de toda a distribuicao posterior. Em problemas pequenos: BayesianRidge e ARDRegression do sklearn implementam inferencia Bayesiana eficiente.

6. Rigorous Model Evaluation via Cross-Validation

Cross-Validation é o conjunto de técnicas para **estimar o erro de generalização** de forma confiável, sem contaminar o conjunto de teste. Princípio fundamental: **nunca use o conjunto de teste para tomar decisões** — nem para escolher lambda, nem para comparar modelos. **Analogia escolar:** dados de treino = lista de exercícios; validação = simulado (ajusta estratégia); teste = prova final (avalia uma única vez).

K-Fold Cross-Validation (método mais usado na prática):

$$CV_K = \frac{1}{K} \sum_{k=1}^K MSE_k$$

K=5 ou K=10 são os valores mais usados. K pequeno: rápido. K grande: mais confiável.

Metodo	Como funciona / Vantagem	Desvantagem	Quando usar
Hold-Out	Split único (80/20). Rápido — O(1) treinos.	Estimativa instável, depende do split aleatório.	N muito grande (>100k)
K-Fold (K=5-10)	K splits rotacionados, média dos K erros. Balanceado custo/precisão.	K vezes mais lento que Hold-Out.	Padrão para a maioria dos casos
Stratified K-Fold	Como K-Fold mas preserva % de classes em cada fold.	Mesmo custo do K-Fold.	Classificação com classes raras (<10%)
Leave-One-Out	K=N: cada amostra é validada uma vez. Mínimo viés possível.	O(N) treinos — impraticável para N grande.	N muito pequeno (<50-100)

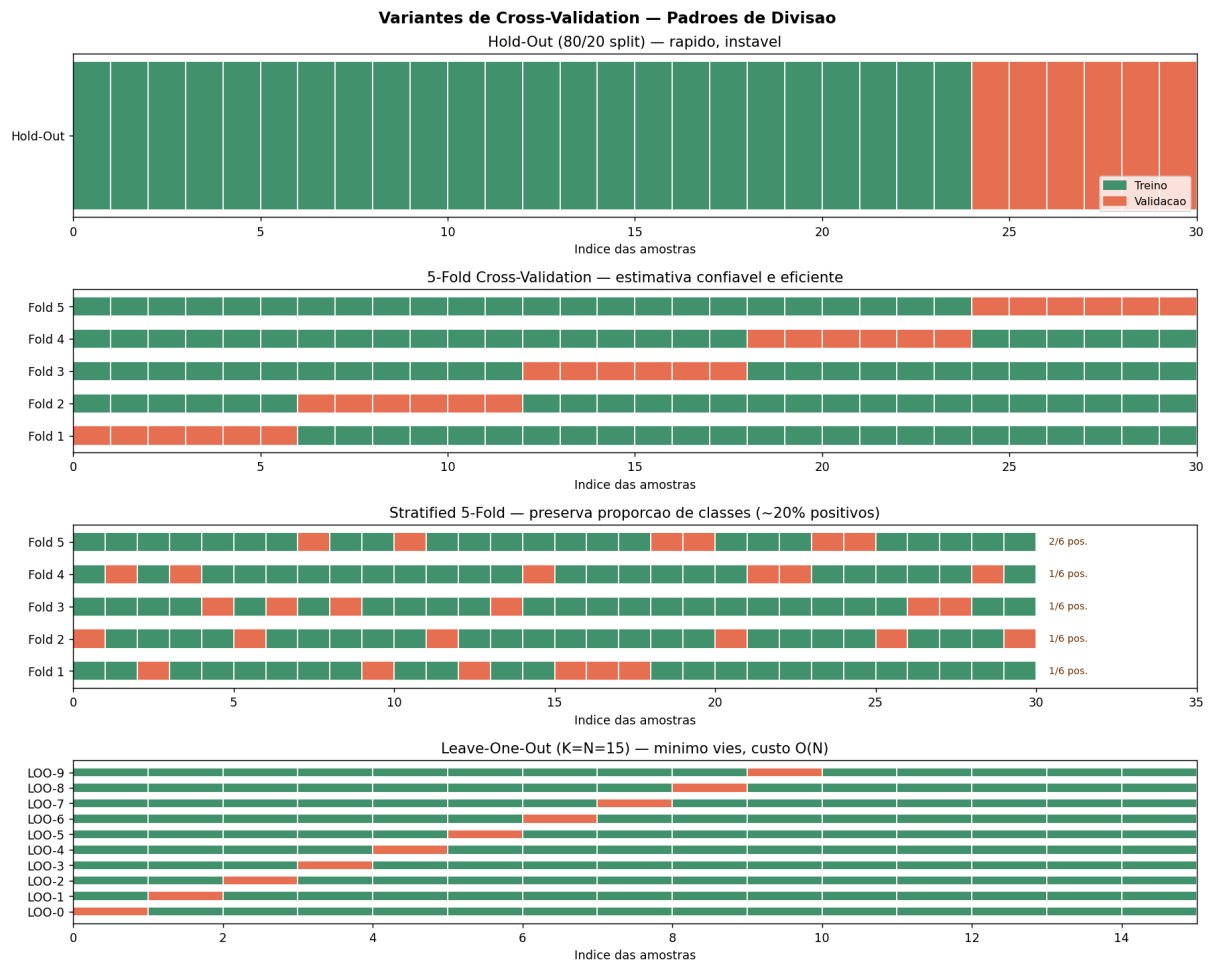


Figura 8 — Padroes de divisao para cada variante de Cross-Validation. Verde: treino. Vermelho: validacao. Hold-Out: um unico split. K-Fold: cada fold e validacao uma vez. Stratified: proporcao de classes preservada em cada fold (indicado pelo texto 'X/Y pos.' a direita). LOO: cada amostra e validacao exatamente uma vez.

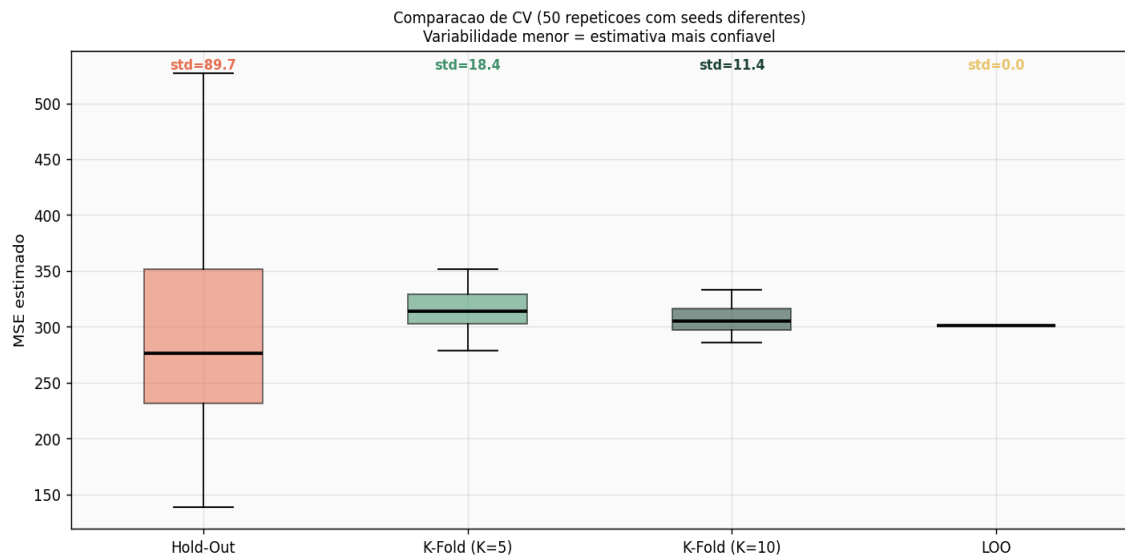


Figura 9 — Comparacao da variabilidade de cada metodo em 50 repeticoes com seeds diferentes. Hold-Out tem a maior variabilidade (desvio padrao alto). K-Fold (K=10) e LOO tem estimativas mais estaveis. LOO e deterministico (mesma estimativa a cada execucao).

Por que isso e importante para Data Science?

A razão pela qual não podemos usar o conjunto de teste para tunar λ e que estariam otimizando para aquele conjunto específico de amostras — vazando informação do teste para o treino. O erro final reportado seria otimisticamente enviesado. Isso é um dos erros mais comuns em competições de ML.

Aplicação Prática em Machine Learning

sklearn: `cross_val_score(modelo, X, y, cv=KFold(5), scoring='neg_mean_squared_error')`. Para classificação desbalanceada: `StratifiedKFold(n_splits=5, shuffle=True)`. Para séries temporais (dados com dependência temporal): `TimeSeriesSplit` em vez de `KFold`.

7. Hyperparameter Tuning & Diagnosis Workflow

O impacto da regularizacao opera em **ordens de magnitude**, nao em escala linear. A diferenca entre $\lambda=0.001$ e $\lambda=0.01$ e enorme (fator 10x), enquanto a diferenca entre $\lambda=0.1$ e $\lambda=0.2$ e minima. **Log-Grid** cobre uniformemente o espaco logaritmico onde o impacto e real:

$$\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 10^0, 10^1, 10^2, 10^3, 10^4, 10^5\}$$

`np.logspace(-5, 5, 20)` — nunca use escala linear para λ !

Estrategia Coarse-to-Fine:

- **Passo 1 — Coarse Sweep:** Log-Grid amplo (10^{-5} a 10^5) para encontrar a **ordem de magnitude certa**.
- **Passo 2 — Fine Sweep:** Log-Grid estreito ao redor do melhor valor encontrado — refina a precisao sem custo computacional proibitivo.
- **Regra de ouro:** usar sempre **K-Fold CV** para avaliar cada valor de λ durante o tuning — **NUNCA** o conjunto de teste.

Estrategia Coarse-to-Fine com Log-Grid
Por que log-scale? Cada salto = ordem de magnitude

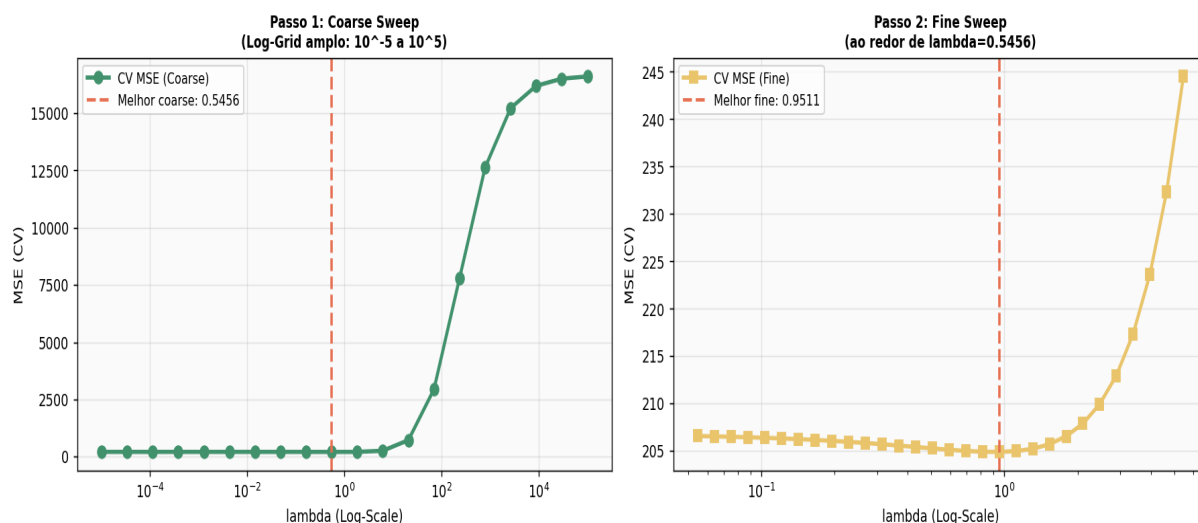


Figura 10 — Estrategia Coarse-to-Fine. Passo 1 (esquerda): sweep amplo identifica que o melhor λ esta na regio de 10^{-1} a 10^1 . Passo 2 (direita): sweep fino ao redor do melhor valor encontrado refina a estimativa sem repetir todo o processo. Escala log-linear e essencial para visibilidade.

Diagnostico via Learning Curves:

Padrao observado	Diagnostico	Acao recomendada
Erro treino baixo, erro val. alto, GAP GRANDE	Under-Regularization (High Variance / Overfitting)	Aumentar λ ; mais dados; reduzir complexidade
Erro treino alto, erro val. alto, GAP PEQUENO	Over-Regularization (High Bias / Underfitting)	Diminuir λ ; mais features; modelo mais complexo

Erro treino aprox. erro val.
aprox. Irred. Error

Otimo — regularizacao bem
calibrada

Manter lambda; focar em features
e dados

Diagnostico: Under vs. Over-Regularization via Learning Curves

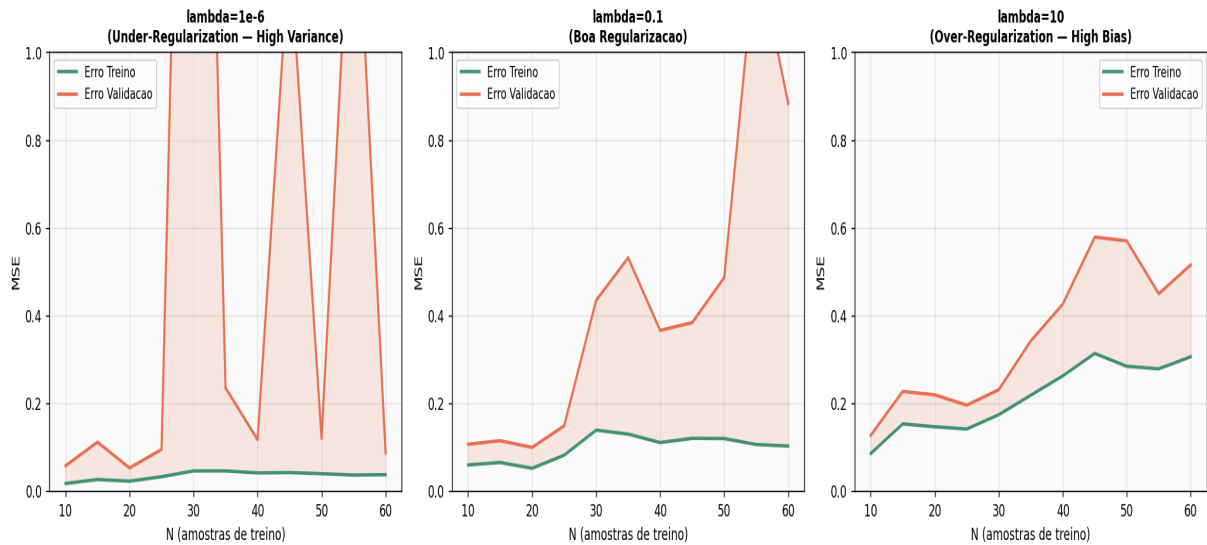


Figura 11 — Learning Curves para tres valores de lambda. Esquerda: lambda muito pequeno causa alto gap entre treino e validacao (Overfitting). Centro: lambda otimo — os dois erros convergem para um valor aceitavel. Direita: lambda muito grande causa ambos os erros elevados (Underfitting — modelo nao aprende nada util).

Por que isso e importante para Data Science?

O Log-Grid e a estrategia Coarse-to-Fine sao o protocolo padrao da industria para tuning de hiperparametros de regularizacao. Usar escala linear desperdicaria a maioria dos pontos avaliados em regioes sem impacto real. O diagnostico via Learning Curves permite identificar QUAL componente (Bias ou Variance) domina o erro — sem isso, qualquer intervencao e cega.

Aplicacao Pratica em Machine Learning

sklearn: `GridSearchCV(Ridge(), param_grid={'alpha': np.logspace(-5,5,20)}, cv=5)`. Para Lasso: `LassoCV(alphas=np.logspace(-5,5,20), cv=5).fit(X,y)` — implementacao eficiente do CV via Coordinate Descent. `RidgeCV` e `LassoCV` sao alternativas altamente otimizadas ao `GridSearchCV` para esses modelos especificos.

8. Resumo Final & Checklist

Tecnica	Penalidade / Mecanismo	Prior Bayesiano	Quando usar
L2 / Ridge	$\lambda * \theta ^2$	Gaussian $N(0, \tau^2)$	Padrao; $p > N$; colinearidade
L1 / Lasso	$\lambda * \theta _1$	Laplace(0, b)	Feature Selection; esparsidade
Elastic Net	$\lambda_1 * \theta _1 + \lambda_2 * \theta ^2$	Laplace + Gaussian	Features correlacionadas
Early Stopping	Parar na epoca t (equiv. L2 com $\lambda=1/\eta * t$)	Implicitamente Gaussian	Deep Learning; qualquer iterativo
Dropout	Mascarar p% dos neuronios (Ensemble implicito)	—	Redes neurais profundas (MLP, Transformer)
Label Smoothing	$y_{smooth} = y * (1 - \epsilon) + \epsilon / K$	—	Classificacao; Transformers; evitar overconfidence

Checklist de Regularizacao em novos projetos:

- Sempre dividir dados em treino / validacao / teste antes de qualquer exploracao.
- Comecar com Ridge (L2) como baseline — simples, robusto, sempre funciona.
- Se interpretabilidade e esparsidade sao prioritarias => tentar Lasso ou Elastic Net.
- Sempre usar Log-Grid para buscar lambda: `np.logspace(-5, 5, 20)` como ponto de partida.
- Usar K-Fold CV ($K=5$ ou $K=10$) para avaliar cada lambda — NUNCA o conjunto de teste.
- Diagnosticar via Learning Curves: gap grande = Variance alta = aumentar lambda; ambos erros altos = Bias alto = diminuir lambda.
- Para Deep Learning: adicionar Dropout($p=0.3-0.5$) e monitorar val loss para Early Stopping.
- NUNCA reportar metricas do conjunto de validacao como resultado final — reservar o teste para uma unica avaliacao no final.

Na proxima aula, aprofundaremos os **metodos baseados em arvores de decisao** (Random Forests, Gradient Boosting) e veremos como os principios de regularizacao desta aula se manifestam em modelos nao-lineares — por exemplo, a profundidade maxima da arvore como Hard Regularization, e o shrinkage do Boosting como Soft Regularization. Os fundamentos de Cross-Validation desta aula sao identicos para avaliar esses modelos.